Apollo Lunar Seismology Tutorial

Reading deep-moonquake stacks and comparing a raw Apollo event

Kwun Hei Fong and Qili Andy Dai

Purpose

This tutorial introduces an undergraduate workflow for Apollo seismic data. You will download an archived deep-moonquake stack, align its three components, inspect a simple polarization weight, and retrieve one raw Apollo 12 waveform from EarthScope. The figures are processing examples, not automatic identifications of lunar core phases.

Data and software

The stacked waveforms and event catalog come from the Nunn et al. lunar seismology archive. The raw event and instrument response come from EarthScope FDSN services. The example uses cluster A1, Apollo 12 station S12, run st9, and channels MHZ, MHX, and MHY.

Install Miniconda or Anaconda, then create a Python environment:

```
conda create -n moon_seismology -c conda-forge \
    python=3.12 obspy numpy scipy matplotlib
conda activate moon_seismology
```

Download the example files

Save the companion script as **lunar_seismic_tutorial.py** in a new working directory. Run the commands below from that directory. The ZIP archive is about 20 MB and expands into a folder named **Lognonne_et_al_2003_deep_moonquake_stacks**.

```
mkdir -p data
curl -L --fail \
  'https://zenodo.org/records/7787610/files/Lognonne_et_al_2003_deep_moonquake_stacks.zip?download=1' \
  -o data/Lognonne_et_al_2003_deep_moonquake_stacks.zip
unzip -q data/Lognonne_et_al_2003_deep_moonquake_stacks.zip -d data
curl -L --fail \
  'https://zenodo.org/records/7787610/files/levent.1008c?download=1' \
  -o data/levent.1008c
```

The complete tested script is also available from the CUHK Seismology Lab tutorial repository. Download the **.py** file, not its HTML preview.

Run and inspect the stacked waveforms

```
python lunar_seismic_tutorial.py
# Optional: also request the single raw Apollo 12 event
python lunar_seismic_tutorial.py --fetch-raw
```

The script writes figures to **figures/**. For A1/S12/st9, the archived MHZ record begins about 35 seconds later than MHX/MHY. The code therefore uses only the common interval and interpolates all three components onto one sampling grid before applying any sample-by-sample operation.

```
start = max(tr.stats.starttime for tr in traces)
end = min(tr.stats.endtime for tr in traces)
sampling_rate = min(tr.stats.sampling_rate for tr in traces)
time = np.arange(int((end - start) * sampling_rate) + 1) / sampling_rate
for tr in traces:
    native_time = (tr.stats.starttime - start
                   + np.arange(tr.stats.npts) / tr.stats.sampling_rate)
    aligned = np.interp(time, native_time, tr.data.astype(float))
```

For this example, the aligned record has 13,139 samples at 6.6251 Hz and begins at 1973-09-30 04:06:44.481 UTC. The three channels are plotted below. Stack time is a property of the processed archive and should not be treated as the origin time of a single event.

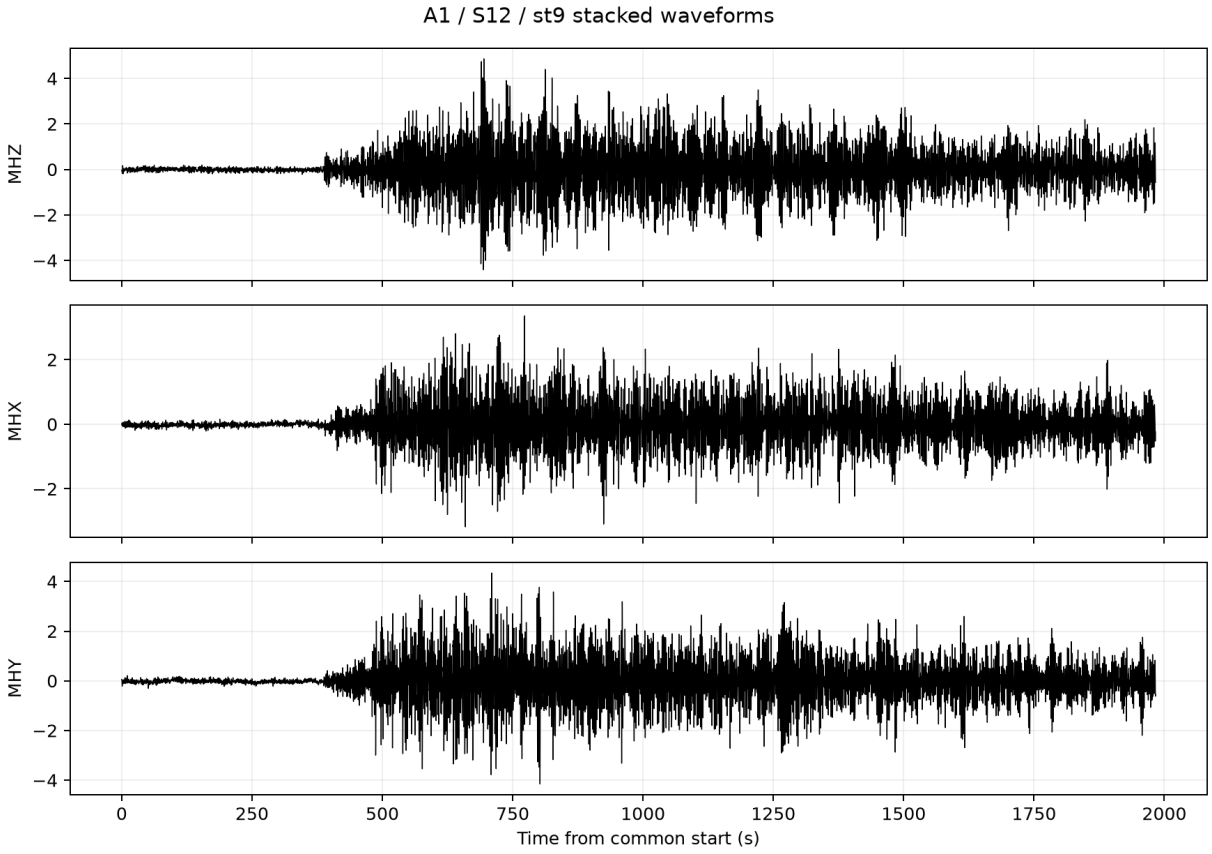


Figure 1. Downloaded and aligned A1/S12/st9 stacked waveforms. Amplitudes are in the units of the archived stack.

A simple polarization weight

After detrending and a 0.05-1.0 Hz bandpass, the script calculates the covariance matrix of the vertical and two horizontal channels in a 10-second moving window. Let its eigenvalues be L1 >= L2 >= L3. A rectilinearity weight is defined as **W = clip[1 - (L2 + L3)/(2 L1), 0, 1]**. The example multiplies the bandpassed vertical record by W.

This operation is independent of the horizontal coordinate direction. The channel names MHX and MHY do not establish radial and transverse orientations; rotation to R and T requires source-station geometry and channel orientation metadata. A large weight indicates locally rectilinear motion, not a uniquely identified P wave, S wave, or core reflection.

```
# After computing the windowed 3 x 3 covariance matrices:
eigenvalues = np.linalg.eigvalsh(covariance)
largest = np.maximum(eigenvalues[:, 2], np.finfo(float).eps)
weight = 1 - (eigenvalues[:, 0] + eigenvalues[:, 1]) / (2 * largest)
weight = np.clip(weight, 0, 1)
weighted_vertical = filtered[0] * weight
```

The companion script contains the complete covariance calculation. Compare the weighted result with the bandpassed trace rather than treating weighting as proof that an arrival is a body wave.

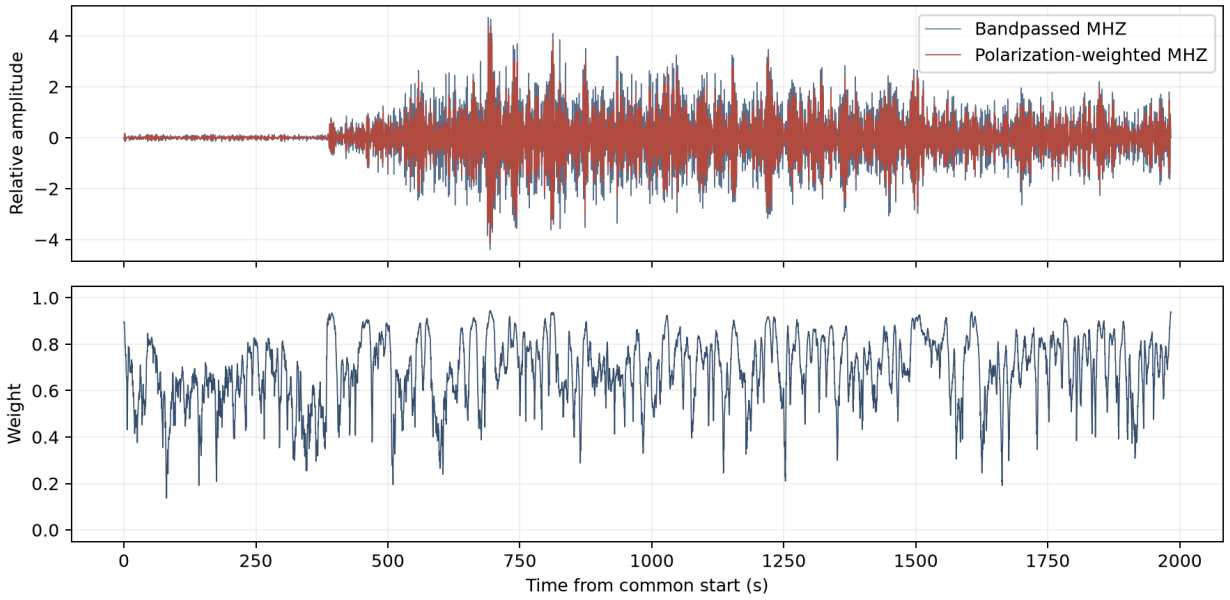


Figure 2. Bandpassed MHZ, polarization-weighted MHZ, and the time-varying weight computed from all three components.

Compare a raw EarthScope event

The optional **--fetch-raw** step requests XA.S12.00.MHZ from 1971-01-04 16:51 to 17:06 UTC. It retrieves the matching station response, removes a mean and linear trend, tapers the waveform, and converts it to displacement. The raw event is much less straightforward to interpret than a multi-event stack.

```
from obspy import UTCDateTime
from obspy.clients.fdsn import Client

start = UTCDateTime('1971-01-04T16:51:00')
client = Client('EARTHSCOPE')
stream = client.get_waveforms('XA', 'S12', '00', 'MHZ', start, start + 900)
inventory = client.get_stations(network='XA', station='S12',
    location='00', channel='MHZ', starttime=start,
    endtime=start + 900, level='response')
trace = stream[0].copy()
trace.detrend('demean')
trace.detrend('linear')
trace.taper(max_percentage=0.05)
trace.remove_response(inventory=inventory, output='DISP',
    pre_filt=(0.001, 0.005, 1.0, 2.0), water_level=60)
```

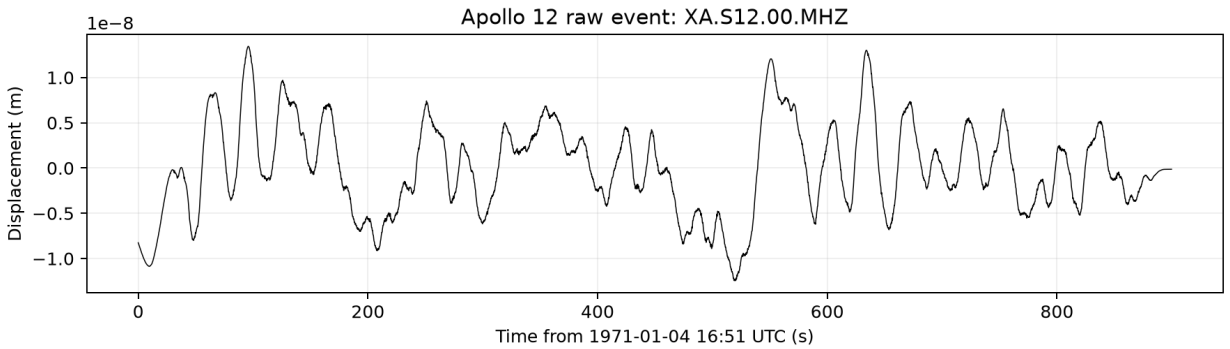


Figure 3. Response-corrected vertical displacement for the selected Apollo 12 event. The displayed 900-second window was downloaded from EarthScope.

Interpretation and limits

The 2003 stack is suitable for learning waveform handling and exploratory polarization analysis. Its processing history, component orientation, and event alignment must be accounted for before quantitative phase picking. Identifying PcP, ScS, or other core-related arrivals requires predicted travel times and comparison across events and stations; the plots alone do not establish those phases.

Sources: Nunn et al., *Lunar Seismology: a Data and Instrumentation Review - Electronic Supplement*, Zenodo, doi:10.5281/zenodo.7787610; EarthScope FDSN data and station services, https://service.earthscope.org/fdsnws/.